INSTITUTO FEDERAL
São Paulo

PHP: estruturas de repetição

Prof. DR. Eder Pansani
epansani@gmail.com / epansani@ifsp.edu.br

Conteúdo da aula

Nesta aula serão abordadas as **<u>estruturas de repetição</u>** (***loops***).

Mas antes disso relembraremos os conceitos mais importantes abordados na última aula.

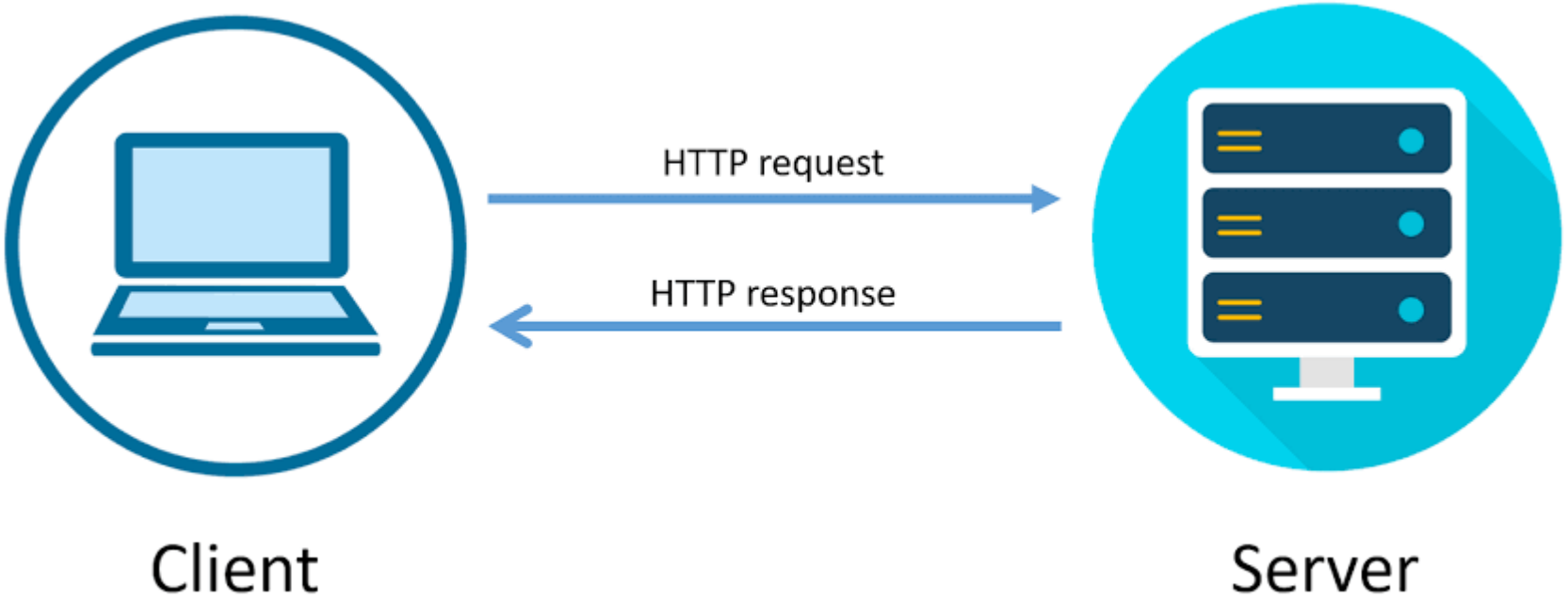
Relembrando...

Prof. DR. Eder Pansani
epansani@gmail.com / epansani@ifsp.edu.br

Relembrando

Na aula anterior começamos com o **Tratamento de requisições** em PHP, processo baseado na **transferência** de **arquivos de texto** via **protocolo HTTP**.

Relembrando



Relembrando

Em seguida discutimos sobre **requisições**, que são compostas por

- Informações da requisição (*request line*)
- Cabeçalhos (*headers*)
- Corpo (*body*)

Relembrando

```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Content-Length: 35

bookId=12345&author=Tan+Ah+Teck
```

- Request Line
- Request Headers
- Request Message Header
- A blank line separates header & body
- Request Message Body

Relembrando

Vimos que uma resposta (response) contém além da versão do protocolo e de um texto de status um **código de status**.

Relembrando

INSTITUTO FEDERAL
São Paulo

HTTP Status Codes

2xx Success	
200	Success / OK

3xx Redirection	
301	Permanent Redirect
302	Temporary Redirect
304	Not Modified

4xx Client Error	
401	Unauthorized Error
403	Forbidden
404	Not Found
405	Method Not Allowed

5xx Server Error	
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

INFIDIGIT

Relembrando

Terminamos falando dos **dois principais métodos**: POST e GET.

Relembrando

Sobre o **método POST**, que em geral é usado quando queremos **enviar dados para o servidor** por meio de um **formulário** HTML.

Relembrando

- Dados no corpo da requisição (“invisíveis” para o usuário)
- Não tem limite (MB)
- Funcionalidades avançadas (*upload*)

Relembrando

Método POST

The HTTP Method

Path to the source on Web Server

Protocol Version Browser supports

```
POST /RegisterStudent.asp HTTP/1.1
Host: guru99.com
User-Agent: Mozilla/5.0
Accept-text/xml,text/html,text/plain, image/jpeg
Accept-Language: en-us,en
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1, utf-8
Keep-Alive: 300
Connection:keep-alive
User=jhon&pass=java
```

The Request Headers

Message body

Guru99.com

Relembrando

Em seguida foi feito um exemplo para tentar demonstrar e explicar de forma prática...

Relembrando

No exemplo criamos um formulário com os campos nome, e-mail e uma cor, que são enviados a uma página de **destino** via **requisição POST**.

Relembrando

INSTITUTO FEDERAL
São Paulo

No exemplo a página destino por sua vez recebe os valores criando variáveis para cada um deles:

```
16    <h1 class="my-3">Destino</h1>
17    <hr>
18    <?php
19    $nome = filter_input(INPUT_POST, "nome", FILTER_SANITIZE_SPECIAL_CHARS);
20    $email = filter_input(INPUT_POST, "email", FILTER_SANITIZE_EMAIL);
21    $cor = filter_input(INPUT_POST, "cor", FILTER_SANITIZE_SPECIAL_CHARS);
```

Relembrando

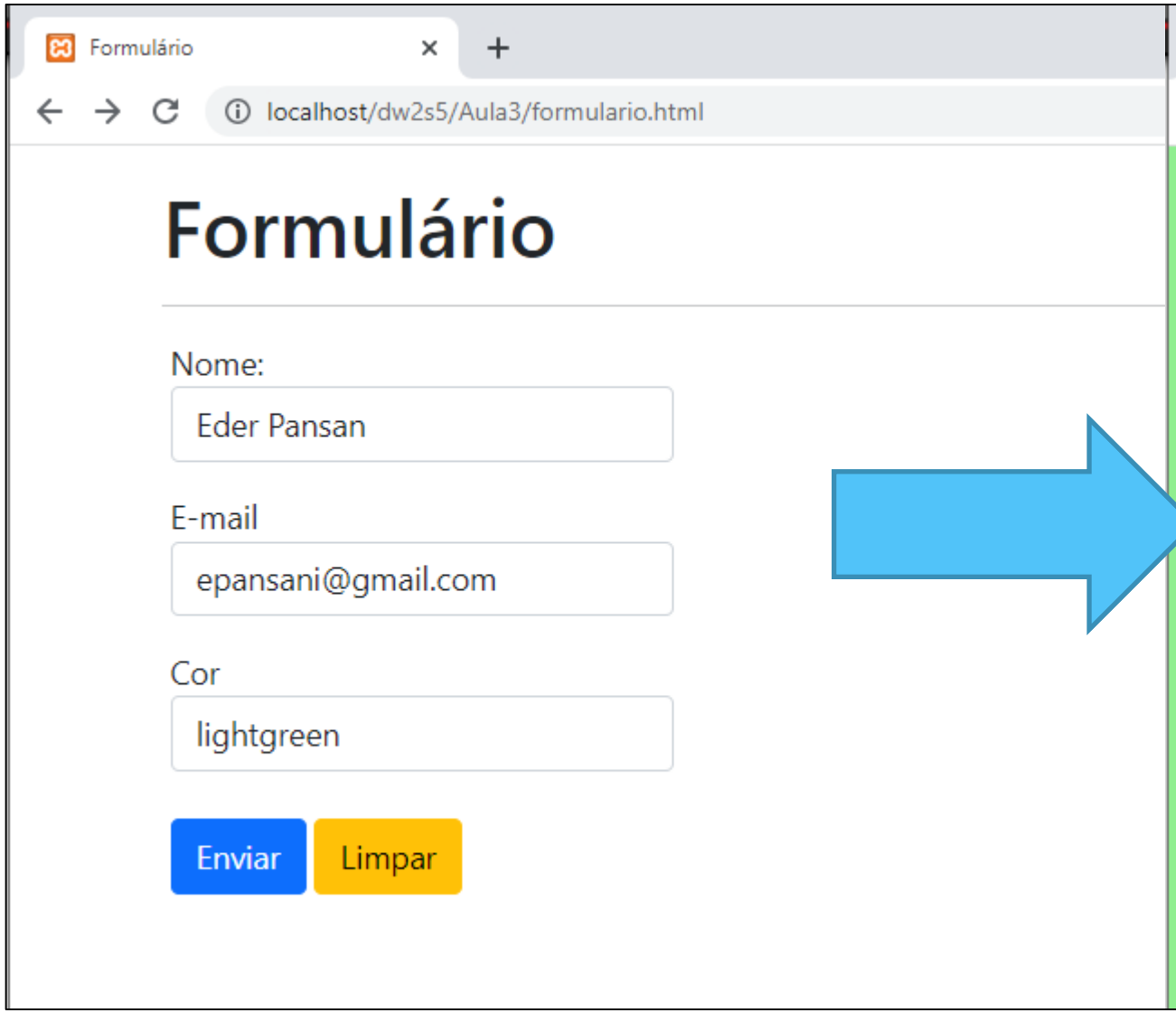
No exemplo na sequência ela mostra (imprime em tela) os valores nome e e-mail (usando echo):

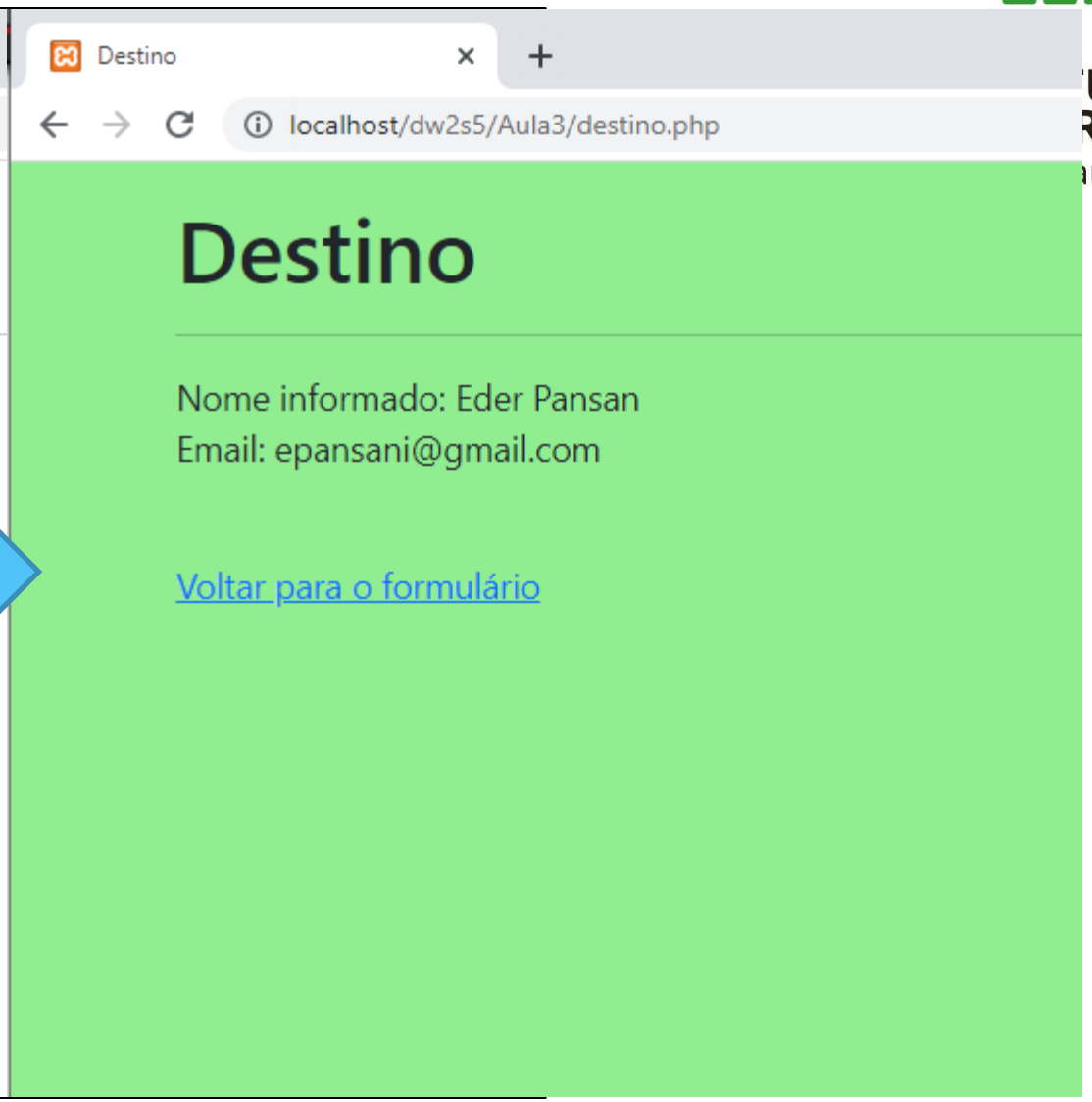
```
22
23      echo "<p>Nome informado: $nome<br>Email: $email</p>";
24      ?>
```

Relembrando

No exemplo por fim o valor “cor” é usado dentro de um trecho de código CSS **(mesclando HTML/CSS e PHP)** para mudar a cor do plano de fundo da página

```
31        <style>
32            body{
33                background-color: <?php echo $cor;?>;
34            }
35        </style>
```


Formulário
localhost/dw2s5/Aula3/formulario.html
Formulário
Nome:
Eder Pansan
E-mail
epansani@gmail.com
Cor
lightgreen
Enviar
Limpar


Destino
localhost/dw2s5/Aula3/destino.php
Destino
Nome informado: Eder Pansan
Email: epansani@gmail.com
Voltar para o formulário

Relembrando

Preste atenção à forma de pegar os valores, informações enviadas em uma requisição.

É preciso observar os "***names***" dos campos (no formulário neste exemplo) e na página destino utilizá-los para pegar as informações.

```html
17        <h1 class="my-3">Formulário</h1>
18        <hr class="mb-4">
19
20        <form class="row g-3" action="destino.php" method="POST">
21            <div class="row">
22                <div class="col-3 my-2">
23                    <label for="nome">Nome:</label>
24                    <input class="form-control" type="text" name="nome" id="nome" required autofocus>
25                </div>
26            </div>
27            <div class="row">
28                <div class="col-3 my-2">
29                    <label for="email">E-mail</label>
30                    <input class="form-control" type="email" name="email" id="email" placeholder="Digite s
31                </div>
32            </div>
33            <div class="row">
34                <div class="col-3 my-2">
35                    <label for="cor">Cor</label>
36                    <input class="form-control" type="text" name="cor" id="cor" required>
37                </div>
```

```
14    <div class="container">
15
16        <h1 class="my-3">Destino</h1>
17        <hr>
18        <?php
19        $nome = filter_input(INPUT_POST, "nome", FILTER_SANITIZE_SPECIAL_CHARS);
20        $email = filter_input(INPUT_POST, "email", FILTER_SANITIZE_EMAIL);
21        $cor = filter_input(INPUT_POST, "cor", FILTER_SANITIZE_SPECIAL_CHARS);
22
23        echo "<p>Nome informado: $nome<br>Email: $email</p>";
24        ?>
25
26        <br>
27        <p>
28            <a href="formulario.html">Voltar para o formulário</a>
29        </p>
30
31        <style>
32            body{
33                background-color: <?php echo $cor;?>;
34            }
35        </style>
```

Relembrando

Importante! o nome do campo da requisição (POST ou GET) **não precisa ser igual ao nome da variável** que receberá o valor. É **recomendado** que seja assim, porém **não é obrigatório**.

```
14      <div class="container">
15
16          <h1 class="my-3">Destino</h1>
17          <hr>
18          <?php
19          $nome = filter_input(INPUT_POST, "nome", FILTER_SANITIZE_SPECIAL_CHAR
20          $email = filter_input(INPUT_POST, "email", FILTER_SANITIZE_EMAIL);
21          $cor = filter_input(INPUT_POST, "cor", FILTER_SANITIZE_SPECIAL_CHARS)
22
23          echo "<p>Nome informado: $nome<br>Email: $email</p>";
24          ?>
25
26          <br>
27          <p>
28              <a href="formulario.html">Voltar para o formulário</a>
29          </p>
30
31          <style>
32              body{
```

Relembrando

Na sequência foi abordado o **método** HTTP **GET**, que **geralmente** é utilizado quando queremos **solicitar um recurso** ao servidor.

Relembrando

Diferente do POST — que normalmente é usado para **enviar dados para processamento** — o GET é mais associado à ideia de **buscar, consultar ou acessar informações**.

Relembrando

- Método padrão
- Dados são enviados de forma visível (no URL)
- Separa o endereço da URL dos dados com um ponto de interrogação.
- É possível marcar a página como favorito.

Relembrando

Método GET

The HTTP Method

Path to the source on Web Server

Parameters to the server

Protocol Version Browser supports

```
GET /RegisterStudent.asp?user=jhon&pass=java HTTP/1.1
Host: guru99.com
User-Agent: Mozilla/5.0
Accept-text/xml,text/html,text/plain, image/jpeg
Accept-Language: en-us,en
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1, utf-8
Keep-Alive: 300
Connection: keep-alive
```

The Request Headers

Guru99.com

Relembrando

O conteúdo também foi trabalhado comum **exemplo** que depois foi aprimorado em um **exercício usando o método GET**, que faz uso da **URL para enviar parâmetros para a página PHP**.

Relembrando

Um dos objetivos foi demonstrar que ao utilizar o método GET **não são necessárias duas páginas**, pode-se enviar e receber parâmetros em uma única página PHP.

Relembrando

Em seguida foram abordados os **operadores** em PHP, começando pelos aritméticos, de atribuição, comparação, lógicos, de *string*, incremento e decremento.

Relembrando

INSTITUTO FEDERAL
São Paulo

Foram abordadas as **<u>estruturas condicionais</u>** (IF, ELSE, SWITCH), sua sintaxe em PHP, exemplos e alguns exercícios para colocar em prática e testar seu funcionamento.

Relembrando

Retomando...

INSTITUTO FEDERAL
São Paulo

Estruturas de repetição *WHILE, FOR, FOREACH*

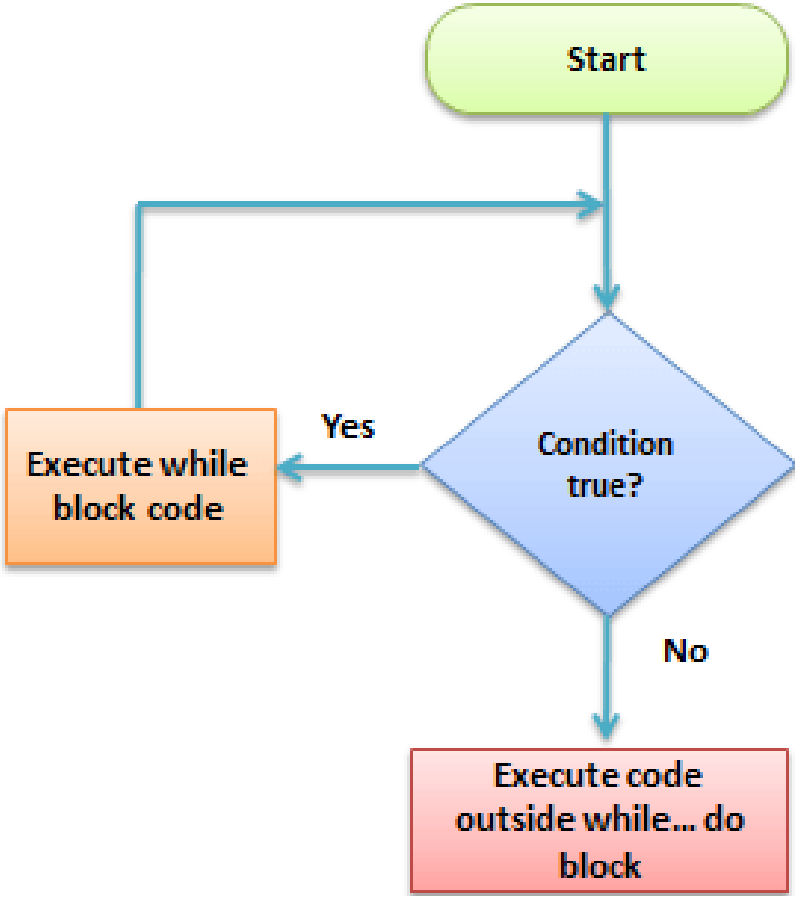
Estruturas de repetição

Os ***loops*** são usados para **executar o mesmo bloco de código repetidamente**, desde que uma **determinada condição seja verdadeira**.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Diagrama de um *loop*



Estruturas de repetição

Muitas vezes precisamos que o mesmo bloco de código seja executado **repetidamente** por um determinado número de vezes. Assim, em vez de adicionar várias linhas de código quase iguais em um script, podemos usar ***loops***.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

PHP suporta as seguintes tipos de *loops*:

- ***while*** - percorre um bloco de código enquanto a condição especificada for verdadeira
- ***do...while*** - percorre um bloco de código uma vez e, em seguida, repete o *loop* enquanto a condição especificada for verdadeira

Estruturas de repetição

- ***for*** - percorre um bloco de código um número especificado de vezes
- ***foreach*** - percorre um bloco de código para cada elemento em uma matriz

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

O *loop* ***while*** executa um bloco de código enquanto a condição especificada for verdadeira.

```
while (condition is true) {
  code to be executed;
}
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

```php
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

O *loop* ***do-while*** sempre executará o bloco de código uma vez, verificará a condição e repetirá o loop enquanto a condição especificada for verdadeira.

```
do {
  code to be executed;
} while (condition is true);
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

```php
<?php
$x = 1;

do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Em um *loop* ***do-while***, a condição é testada **APÓS** a execução das instruções dentro do *loop*. Isso significa que o ***do-while*** executará suas instruções pelo menos uma vez, mesmo que a condição seja falsa.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

```php
<?php
$x = 6;

do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>
```

Estruturas de repetição

Exemplo:

https://www.w3schools.com/php/phptryit.asp?filename=tryphp_loop_do_while2

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Este comportamento é diferente do *loop* ***while***, que testa a condição **ANTES** da execução das instruções dentro do *loop*, sendo assim ele não executará nenhuma vez caso a condição seja falsa.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Observe, link

Run ›

Result Size: 544 x 399

Get your own website

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 6;

do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>

</body>
</html>
```

The number is: 6

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Observe, link

Run ›

Result Size: 544 x 399

Get your own website

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = 6;

while ($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>

</body>
</html>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

O *loop* ***for*** percorre um bloco de código um número especificado de vezes.

```
for (init counter; test counter; increment counter) {
  code to be executed for each iteration;
}
```

Estruturas de repetição

O *loop* ***for*** é mais usado/indicado quando você sabe com antecedência quantas vezes o script deve ser executado.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Exemplo,

```php
<?php
for ($x = 0; $x <= 10; $x++) {
  echo "The number is: $x <br>";
}
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

É possível criar um *loop* ***for*** com incremento (ou decremento) diferente de 1.

```php
<?php
for ($x = 0; $x <= 100; $x+=10) {
  echo "The number is: $x <br>";
}
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

O *loop* ***foreach*** percorre um bloco de código <u>para **cada elemento** em uma **matriz**</u>.

```
foreach ($array as $value) {
  code to be executed;
}
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

O *loop* ***foreach*** funciona apenas em *arrays* e é usado para percorrer cada par chave/valor em um *array*.

Para cada iteração do *loop*, o valor do elemento atual do *array* é atribuído a ***$value*** e o ponteiro do ***array*** é movido por um, até atingir o último elemento do ***array***.

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Exemplo (apresentando somente valores)

```php
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
  echo "$value <br>";
}
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Exemplo (apresentando somente valores)

```php
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $val) {
  echo "$x = $val<br>";
}
?>
```

Estruturas de repetição

INSTITUTO FEDERAL
São Paulo

Referências e leitura complementar:

https://www.w3schools.com/php/php_looping.asp

https://www.w3schools.com/php/php_looping_while.asp

https://www.w3schools.com/php/php_looping_do_while.asp

https://www.w3schools.com/php/php_looping_for.asp

https://www.w3schools.com/php/php_looping_foreach.asp

INSTITUTO FEDERAL
São Paulo

Dúvidas?

Perguntas?

Sugestões?